



PandaGestion

Finanzas personales

Arquitectura y circuito completo de funcionamiento

Guía técnica detallada de las capas que intervienen desde que el usuario ingresa a una opción hasta que consulta, crea, modifica, paga o elimina información.

Tecnologías: Next.js 16 · React 19 · JavaScript · Tailwind CSS · SweetAlert2 · SQLite · Docker

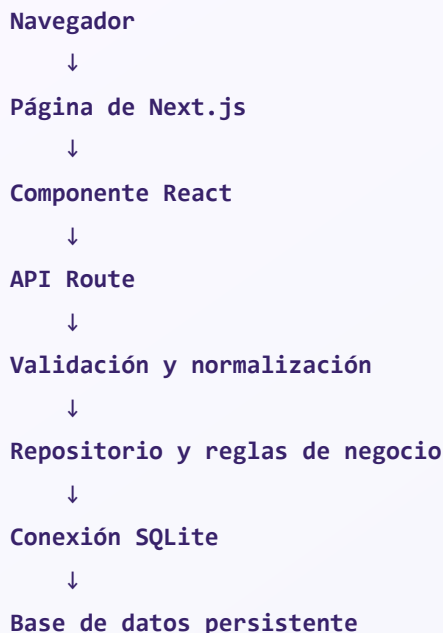
Documento: versión 1.0 · 7 de septiembre de 2026

Contenido

1. Visión general
2. Capas principales
3. Ingreso a una opción
4. Carga inicial de información
5. API y consultas GET
6. Inicialización de SQLite
7. Creación de un concepto
8. Validación y normalización
9. Persistencia mediante repositorios
10. Edición y eliminación
11. Administración de categorías
12. Gastos mensuales
13. Separación entre conceptos y gastos
14. Pagos, filtros y ordenamiento
15. Dashboard e historial
16. Manejo de errores
17. Funcionamiento en Docker
18. Responsabilidades por archivo

1. Visión general

PandaGestion está organizada en capas para que la presentación, la comunicación HTTP, las validaciones, las reglas de negocio y la persistencia no queden mezcladas. Cada capa recibe información de la anterior, realiza una tarea concreta y entrega un resultado a la siguiente.



Cuando una operación termina, el resultado recorre el camino inverso: SQLite devuelve filas, el repositorio las transforma, la API responde JSON y React actualiza la pantalla.

2. Capas principales

Capa	Responsabilidad	Ubicación
Layout y navegación	Estructura general, menú lateral, navegación móvil y área de contenido.	<code>app/layout.js</code> <code>components/AppShell.js</code>
Páginas	Relacionan una URL con la pantalla que debe mostrarse.	<code>app/**/page.js</code>
Componentes cliente	Formularios, modales, estados React, eventos, filtros y solicitudes HTTP.	<code>components/*Manager.js</code>
API	Recibe peticiones HTTP, llama validaciones y repositorios, y devuelve JSON.	<code>app/api/**/route.js</code>
Validación	Limpia, convierte y comprueba los datos externos.	<code>lib/validation.js</code>
Repositorios	Reglas de negocio, SQL, transacciones y transformación de filas.	<code>lib/repositories/</code>

Capa	Responsabilidad	Ubicación
Base de datos	Conexión, configuración, creación del esquema y migraciones.	<code>lib/db.js</code>
Persistencia	Archivo físico que conserva conceptos, gastos y categorías.	<code>data/panda-gestion.db</code> o volumen Docker
Infraestructura	Construcción de la imagen, proceso Node y volumen persistente.	<code>Dockerfile</code> <code>docker-compose.yml</code>

3. Qué ocurre al ingresar a una opción

Supongamos que el usuario abre `http://servidor:3000/conceptos`.

3.1 El navegador solicita la URL

```
GET /conceptos
```

Next.js busca automáticamente `app/conceptos/page.js`. El nombre `page.js` es una convención obligatoria del App Router: ese archivo convierte la carpeta `conceptos` en una ruta navegable.

```
export default function ConceptsPage() {
  return <ConceptsManager />;
}
```

3.2 Se aplica el layout general

Todas las páginas pasan por `app/layout.js`:

```
<AppShell>{children}</AppShell>
```

`children` representa la pantalla pedida. `AppShell` agrega el menú, el logo y el contenedor principal. Los enlaces de Next.js cambian de opción sin recargar por completo el sitio.

3.3 Componentes de servidor y de cliente

Los archivos `page.js` son componentes de servidor por defecto. Los administradores interactivos comienzan con:

```
"use client";
```

Esta directiva permite utilizar `useState`, `useEffect`, eventos, formularios, modales, SweetAlert2 y `fetch` en el navegador.

4. Carga inicial de información

Al montarse `ConceptsManager`, React crea estados locales:

```
const [concepts, setConcepts] = useState([]);
const [categories, setCategories] = useState([]);
const [loading, setLoading] = useState(true);
const [open, setOpen] = useState(false);
const [editing, setEditing] = useState(null);
const [form, setForm] = useState(emptyForm);
```

Estado	Uso
<code>concepts</code>	Lista obtenida del servidor.
<code>categories</code>	Categorías disponibles y colores.
<code>loading</code>	Determina si se muestra el indicador de carga.
<code>open</code>	Controla la apertura del modal.
<code>editing</code>	Indica si se crea un registro o se edita uno existente.
<code>form</code>	Contiene los valores actuales de los campos.

`useEffect` llama a `load()` después de montar la pantalla. Se realizan dos peticiones en paralelo:

```
Promise.all([
  fetch("/api/concepts", { cache: "no-store" }),
  fetch("/api/categories", { cache: "no-store" })
]);
```

`no-store` evita reutilizar una respuesta vieja. La interfaz siempre solicita el estado actual de SQLite.

5. API y consultas GET

La petición `GET /api/concepts` se relaciona con `app/api/concepts/route.js`. En los archivos `route.js`, `Next.js` reconoce funciones llamadas `GET`, `POST`, `PUT`, `PATCH` y `DELETE`.

```
export function GET() {
  return NextResponse.json({ data: listConcepts() });
}
```

La API no contiene HTML ni SQL extenso. Actúa como controlador: recibe HTTP, llama a las capas internas y construye una respuesta.

5.1 El repositorio consulta SQLite

```
SELECT *  
FROM concepts  
ORDER BY is_active DESC, name COLLATE NOCASE;
```

SQLite devuelve nombres adaptados a la base:

```
{  
  id: 1,  
  name: "Internet",  
  category: "Servicios",  
  is_active: 1  
}
```

El repositorio los transforma a un objeto apropiado para JavaScript:

```
{  
  id: 1,  
  name: "Internet",  
  category: "Servicios",  
  isActive: true  
}
```

5.2 La respuesta regresa a React

```
{  
  "data": [  
    {  
      "id": 1,  
      "name": "Internet",  
      "category": "Servicios",  
      "isActive": true  
    }  
  ]  
}
```

El componente ejecuta `setConcepts(...)`, `setCategories(...)` y `setLoading(false)`. React vuelve a renderizar y reemplaza el indicador de carga por la grilla.

6. Inicialización y migración de SQLite

Los repositorios solicitan la conexión mediante `getDatabase()`, definida en `lib/db.js`. La conexión es diferida: importar un repositorio no abre la base; se abre al ejecutar la primera operación real.

Este diseño evita que los workers paralelos de `next build` intenten configurar el mismo archivo mientras se construye la imagen Docker.

1. Calcula la ruta del archivo.
2. Crea el directorio si no existe.
3. Abre SQLite mediante `better-sqlite3`.
4. Configura espera, WAL y claves foráneas.
5. Crea tablas e índices faltantes.
6. Ejecuta migraciones de columnas.
7. Inserta las categorías iniciales cuando corresponde.
8. Conserva la conexión para reutilizarla.

```
database.pragma("busy_timeout = 5000");
database.pragma("journal_mode = WAL");
database.pragma("foreign_keys = ON");
```

Configuración	Finalidad
<code>busy_timeout</code>	Espera hasta cinco segundos si otra operación está utilizando la base.
<code>journal_mode = WAL</code>	Mejora la convivencia entre lecturas y escrituras.
<code>foreign_keys = ON</code>	Activa la integridad de las relaciones entre tablas.

Las tablas se crean con `CREATE TABLE IF NOT EXISTS`. Las instalaciones anteriores se actualizan con una comprobación como:

```
ensureColumn(database, "expenses", "closing_date", "TEXT");
dropColumn(database, "concepts", "default_amount_cents");
dropColumn(database, "concepts", "due_day");
dropColumn(database, "concepts", "closing_day");
```

Resultado: actualizar el contenedor no borra la información. La aplicación detecta y agrega la estructura que falta.

7. Circuito completo para crear un concepto

```
Clic en "Nuevo concepto"
↓
Estado React abre el modal
↓
Usuario completa campos controlados
↓
Validación nativa de HTML
↓
save() serializa el formulario
↓
POST /api/concepts
↓
parseConcept()
↓
createConcept()
↓
INSERT en SQLite
↓
HTTP 201 + JSON
↓
Recarga de grilla + SweetAlert2
```

7.1 Apertura del modal

`showCreate()` limpia cualquier edición anterior, prepara un formulario vacío, selecciona la primera categoría activa y cambia `open` a `true`. El modal se renderiza mediante un portal directamente en `document.body`, evitando recortes por contenedores animados.

7.2 Campos controlados

```
<input
  value={form.name}
  onChange={(event) =>
    setForm({ ...form, name: event.target.value })
  }
/>
```

El valor visible procede del estado. Cada pulsación ejecuta `onChange`, actualiza `form` y provoca un nuevo render.

7.3 Primera barrera: validación HTML

Los atributos `required` y `maxLength` permiten al navegador detectar campos vacíos y longitudes excesivas antes de enviar.

Importante: la validación HTML mejora la experiencia, pero no brinda seguridad por sí sola. Una petición puede construirse manualmente, por lo que el servidor siempre vuelve a validar.

7.4 Envío desde el navegador

```
async function save(event) {
  event.preventDefault();

  await fetch("/api/concepts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(form)
  });
}
```

`preventDefault()` evita una recarga completa. El formulario se convierte en JSON y se envía a la API.

8. Validación y normalización

El endpoint ejecuta:

```
const concept = createConcept(
  parseConcept(await request.json())
);
```

El orden exacto es: leer JSON, ejecutar `parseConcept` y, solamente si no hay errores, llamar a `createConcept`.

`parseConcept` no se limita a responder verdadero o falso. También normaliza:

```
Entrada del navegador:
{
  name: " Internet ",
  category: "Servicios",
  isActive: true
}

Salida validada:
{
  name: "Internet",
  category: "Servicios",
  isActive: 1
}
```

El concepto no contiene importes ni fechas. Esos valores se solicitan y validan al crear cada gasto. Los importes de los gastos se almacenan como centavos enteros para evitar errores de coma flotante y se muestran siempre con exactamente dos decimales: `$12.500,50` se guarda como `1250050`.

Si una regla no se cumple, la función lanza una excepción:

```
throw new Error(  
    "Ingresá un nombre válido (máximo 80 caracteres)."  
);
```

La excepción interrumpe el circuito antes de llegar a SQLite.

9. Persistencia mediante repositorios

El repositorio comprueba reglas que requieren consultar la base. Antes de crear un concepto verifica que la categoría exista y esté activa:

```
SELECT id  
FROM categories  
WHERE name = ? AND is_active = 1;
```

Después utiliza una consulta preparada:

```
INSERT INTO concepts (name, category, is_active)  
VALUES (@name, @category, @isActive);
```

Las consultas preparadas separan el SQL de los valores del usuario y reducen el riesgo de inyección. SQLite devuelve el identificador creado; el repositorio consulta el registro y lo devuelve con el formato utilizado por JavaScript.

9.1 Respuesta y actualización visual

La API devuelve **HTTP 201 Created** junto con `{ "data": ... }`. El componente cierra el modal, vuelve a ejecutar `load()`, actualiza la grilla y muestra la confirmación de SweetAlert2.

10. Edición y eliminación de conceptos

10.1 Editar

El lápiz ejecuta `showEdit(concept)`. La función guarda el objeto seleccionado en `editing`, copia sus datos a `form` y abre el modal.

Al guardar, `editing` hace que el método sea `PUT`:

```
PUT /api/concepts/15
```

La carpeta dinámica `[id]` permite obtener `15` desde `context.params`. La API vuelve a utilizar `parseConcept` y luego llama a `updateConcept`.

```
Formulario → PUT → parseConcept → updateConcept  
→ UPDATE SQL → JSON → recarga de grilla
```

Si cambia el nombre, el repositorio actualiza también `expenses.concept_name` para todos los gastos relacionados. Ambas operaciones se ejecutan dentro de una transacción: se confirman juntas o se revierten juntas. Los importes, fechas, notas y estados históricos no cambian.

10.2 Eliminar o archivar

SweetAlert2 pide confirmación y luego se envía:

```
DELETE /api/concepts/15
```

El repositorio comprueba si existen gastos asociados. Si no existen, elimina. Si hay historial, cambia `is_active` a cero. El archivado preserva las referencias históricas y evita que el concepto aparezca al cargar nuevos gastos.

11. Administración de categorías

`CategoriesManager` utiliza los siguientes endpoints:

Método	Ruta	Acción
GET	<code>/api/categories</code>	Listar categorías.
POST	<code>/api/categories</code>	Crear categoría.
PUT	<code>/api/categories/:id</code>	Editar nombre, color y estado.
DELETE	<code>/api/categories/:id</code>	Eliminar o archivar.

`parseCategory` valida nombre, longitud, color hexadecimal y estado. El repositorio evita nombres duplicados.

11.1 Renombrado transaccional

Al renombrar se actualizan la categoría y los conceptos que la utilizan dentro de una transacción. Ambas operaciones se completan juntas o se revierten juntas.

La categoría de los gastos históricos no cambia: cada gasto conserva una fotografía de la categoría utilizada en ese período.

11.2 Eliminación segura

Una categoría sin conceptos puede eliminarse. Si está en uso se archiva, deja de ofrecerse para nuevos conceptos y conserva la coherencia de los existentes.

12. Circuito de gastos mensuales

La ruta puede incluir un período:

```
/gastos?month=2026-09
```

`app/gastos/page.js` valida el parámetro y entrega `initialMonth` a `ExpensesManager`. El componente carga en paralelo:

```
GET /api/expenses?month=2026-09
GET /api/concepts
GET /api/categories
```

Los conceptos alimentan el selector del modal. Las categorías se utilizan para representar sus colores en la grilla; la categoría del gasto proviene del concepto seleccionado.

12.1 Agregar gasto

Al elegir un concepto, el usuario carga los datos propios de ese gasto:

- Importe real del período.
- Fecha de cierre opcional.
- Fecha de vencimiento obligatoria.

```
POST /api/expenses

{
  "conceptId": 8,
  "month": "2026-09",
  "amount": 250000,
  "closingDate": "2026-08-28",
  "dueDate": "2026-09-05",
  "notes": ""
}
```

`parseExpense` valida el identificador, el mes, el importe y las fechas. El repositorio consulta el concepto y toma de él el nombre y la categoría.

La restricción `UNIQUE(concept_id, month)` evita registrar dos veces el mismo concepto en un mismo período.

12.2 Editar gasto

```
PATCH /api/expenses/:id
```

Permite modificar importe, cierre, vencimiento y notas. Se usa `PATCH` porque se actualiza parte del recurso, no necesariamente su identidad completa.

12.3 Eliminar gasto

Después de confirmar con SweetAlert2, `DELETE /api/expenses/:id` elimina solamente la instancia del mes. El concepto reutilizable permanece disponible.

13. Separación entre conceptos y gastos

Los conceptos contienen solamente nombre, categoría y estado. Funcionan como una lista administrable para identificar qué se está pagando.

El importe, la fecha de cierre y el vencimiento pertenecen exclusivamente al gasto mensual. No se completan desde el concepto ni existe una generación automática del mes.

Concepto: Tarjeta de crédito + Tarjetas

↓ selección manual

Gasto de septiembre: importe + cierre + vencimiento + notas

Esta separación evita que un valor estimado se confunda con el importe real. Cada período conserva exactamente la información ingresada por el usuario.

14. Pagos, filtros, días y ordenamiento

14.1 Marcar pagado

```
PATCH /api/expenses/:id
{ "status": "paid" }
```

El repositorio establece `status = 'paid'` y guarda `paid_at`. Al regresar a pendiente, elimina la fecha de pago.

14.2 Filtros y búsqueda

La lista ya cargada en React se filtra por estado y por coincidencia con concepto o categoría. No se consulta nuevamente SQLite porque el volumen de un único mes es pequeño.

14.3 Días restantes

El navegador compara el vencimiento con la fecha local y muestra valores como:

15 días · 1 día · Vence hoy · 2 días vencido · Pagado

14.4 Ordenamiento

`sortBy` y `sortDirection` controlan el orden por concepto, fecha de cierre o vencimiento. Las fechas de cierre vacías permanecen al final.

15. Dashboard e historial

El dashboard consulta en paralelo:

```
GET /api/stats?month=2026-09
GET /api/categories
```

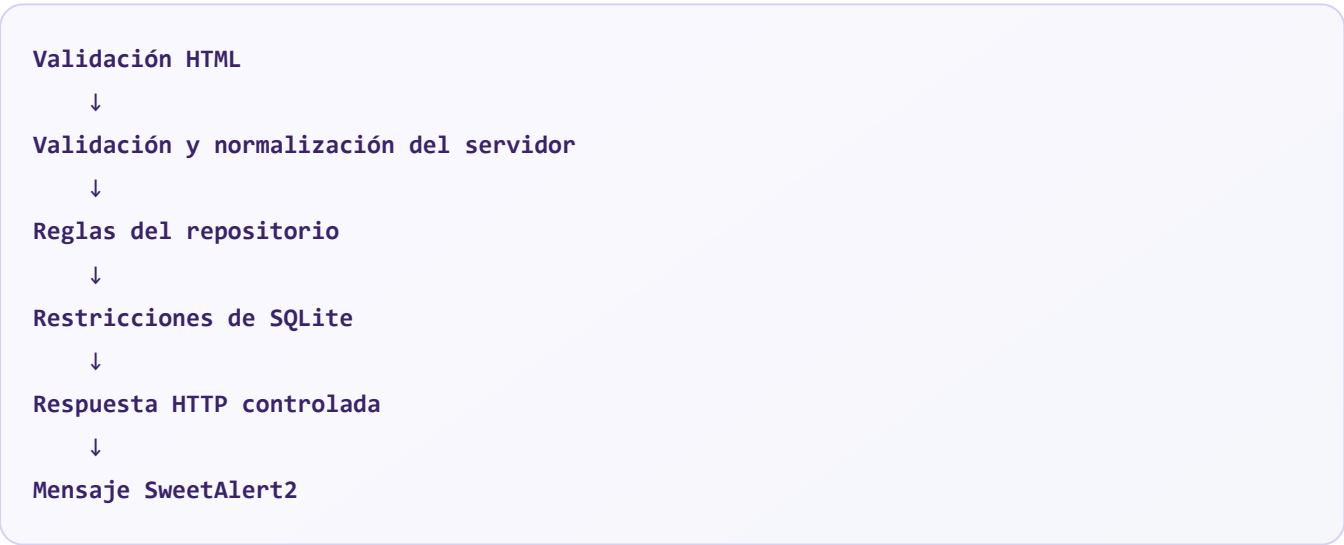
`getStats()` realiza agregaciones SQL:

```
SUM(amount_cents)
SUM(CASE WHEN status = 'paid' THEN amount_cents ELSE 0 END)
SUM(CASE WHEN status = 'pending' THEN amount_cents ELSE 0 END)
```

Se calculan total mensual, total pagado, pendiente, porcentaje de cumplimiento, distribución por categoría, evolución de seis meses y movimientos recientes. Recharts representa esos datos visualmente.

El historial consulta `GET /api/history`, agrupa por mes y compara cada período con el anterior.

16. Manejo de errores



Nivel	Ejemplos	Resultado
HTML	Obligatorio vacío, número fuera de rango.	El navegador detiene el envío.
<code>validation.js</code>	Importe negativo, fecha inválida, nombre demasiado largo.	Se lanza un error y la API responde 400.
Repositorio	Categoría inexistente, concepto inexistente.	La regla de negocio rechaza la operación.
SQLite	Concepto duplicado en el mismo mes.	La restricción protege la integridad.
Frontend	Respuesta <code>{ "error": "..."} .</code>	SweetAlert2 muestra un mensaje entendible.

Los endpoints están envueltos en `try/catch`. Ante un error, no devuelven una página rota, sino JSON y un código HTTP apropiado.

17. Funcionamiento dentro del VPS y Docker

```

Navegador del usuario
    ↓
Dominio o IP del VPS
    ↓
Puerto publicado por Docker
    ↓
Contenedor "pandagestion"
    ↓
Servidor Node.js / Next.js
    ↓
API Route → Repositorio → SQLite
    ↓
/app/data/panda-gestion.db
    ↓
Volumen "pandagestion_data"
```

La imagen contiene el código compilado. El archivo SQLite está fuera de la capa descartable del contenedor, dentro de un volumen:

```
volumes:
  - pandagestion_data:/app/data
```

Por eso `docker compose up -d --build` puede reemplazar el contenedor sin perder datos.

No ejecutar: `docker compose down -v`. La opción `-v` elimina el volumen de SQLite.

`restart: unless-stopped` hace que Docker reinicie la aplicación después de una caída o un reinicio del VPS, salvo que haya sido detenida intencionalmente.

La aplicación se ejecuta con un usuario sin privilegios dentro del contenedor y expone un healthcheck que consulta la API para comprobar su estado.

18. Responsabilidad resumida por archivo

Archivo o carpeta	Responsabilidad
<code>app/layout.js</code>	Estructura común y metadatos.

Archivo o carpeta	Responsabilidad
<code>components/AppShell.js</code>	Menú y navegación general.
<code>app/**/page.js</code>	Punto de entrada de cada URL.
<code>components/*Manager.js</code>	Interacción, formularios, estado y llamadas HTTP.
<code>components/ui/</code>	Modal, cabecera, selector de mes, carga y estados vacíos reutilizables.
<code>app/api/**/route.js</code>	Controladores HTTP y códigos de respuesta.
<code>lib/validation.js</code>	Validación y transformación de entradas.
<code>lib/repositories/</code>	Reglas de negocio, SQL y transacciones.
<code>lib/db.js</code>	Conexión, esquema, configuración y migraciones.
<code>lib/constants.js</code>	Categorías iniciales y colores predeterminados.
<code>lib/formatters.js</code>	Presentación de moneda, meses y fechas.
<code>Dockerfile</code>	Construcción de la imagen optimizada de producción.
<code>docker-compose.yml</code>	Servicio, puerto, reinicio, seguridad y volumen.
<code>CONTEXTO.md</code>	Decisiones funcionales y técnicas para futuras sesiones.

Conclusión

La separación permite modificar una parte sin reescribir todo el sistema. React se ocupa de la interacción; la API, del protocolo HTTP; `validation.js`, de convertir entradas inseguras; los repositorios, de las reglas y consultas; y `db.js`, de administrar SQLite.

Este recorrido consistente se repite para conceptos, categorías, gastos, estadísticas e historial. La consecuencia es un código más legible, comprobable y sencillo de ampliar.